

ADVANCED ENCRYPTION STANDARD (AES)

Implementation

Kiran Deep Ghosh (Crs2310)

September 17, 2024

Contents

1	Introduction	2
1.1	Overview of Advanced Encryption Standard (AES)	2
1.2	Purpose of the Project	2
2	Implementation Strategy	2
2.1	Adherence to NIST FIPS 197 Standard	2
2.2	Focus on a 128-Bit Key	3
3	Key Components in the Implementation	3
4	Code Structure and Modularity	4
5	Testing and Validation	5
6	Security Considerations	6
7	Optimization Techniques	6
8	Conclusion	7

1 Introduction

1.1 Overview of Advanced Encryption Standard (AES)

The Advanced Encryption Standard (AES) is a widely adopted symmetric encryption algorithm known for its security and efficiency. AES operates on 128-bit data blocks and supports key sizes of 128, 192, and 256 bits. This versatility and robustness against cryptographic attacks make it the standard for securing sensitive data. In this project, we focus on implementing AES with a 128-bit key, strictly following the NIST FIPS 197 standard, which details the necessary transformations—SubBytes, ShiftRows, MixColumns, and AddRoundKey—and the key expansion process. By adhering to the NIST guidelines, our implementation ensures reliability, security, and interoperability with other AES-compliant systems.

1.2 Purpose of the Project

The purpose of this project is to implement the AES encryption algorithm using a 128-bit key in C, based on the NIST FIPS 197 standard. The project aims to translate the theoretical aspects of the AES algorithm, including key expansion, substitution, shifting, mixing, and key addition, into practical code. This implementation demonstrates the application of cryptographic standards in software, emphasizing the importance of adhering to established guidelines for security and interoperability.

2 Implementation Strategy

The AES implementation in this project is designed around the NIST FIPS 197 standard, focusing on the use of a 128-bit key. The strategy ensures a robust and compliant encryption mechanism by closely following the provided pseudocode and guidelines.

2.1 Adherence to NIST FIPS 197 Standard

Our implementation strictly follows the NIST FIPS 197 standard, ensuring compatibility and security. This standard provides detailed instructions for the AES algorithm, including the transformations and key expansion required for the encryption process.

2.2 Focus on a 128-Bit Key

This implementation is tailored for a 128-bit key, involving 10 rounds of encryption. The key size influences the structure of the key expansion and the overall encryption process, making the implementation efficient and secure.

3 Key Components in the Implementation

The implementation is built around several key components essential for AES encryption with a 128-bit key:

1. **State Matrix:** The plaintext is structured into a 4x4 matrix of bytes, known as the state matrix, which undergoes a series of transformations to produce the ciphertext.
2. **Key Expansion:** The 128-bit cipher key is expanded into 11 round keys, following the NIST-specified algorithm to ensure secure key generation for each encryption round.
3. **Core AES Functions:** The implementation includes the following key functions:
 - **SubBytes:** Substitutes each byte in the state matrix using the S-Box, introducing non-linearity.
 - **ShiftRows:** Cyclically shifts the rows of the state matrix, contributing to data diffusion.
 - **MixColumns:** Applies a mathematical transformation to mix the bytes in each column, enhancing security.
 - **AddRoundKey:** XORs the state matrix with the current round key, integrating the key into the encryption process.
4. **Cipher Function:** The main function orchestrates the encryption process, applying 10 rounds of transformations. The final round omits the MixColumns step, aligning with the NIST standard.
5. **Decryption Process:** The decryption mirrors the encryption process in reverse:
 - **Initial Round:** XORs the ciphertext with the final round key.
 - **Inverse Main Rounds:** Reverses the encryption steps, including Inverse ShiftRows, Inverse SubBytes, and Inverse MixColumns.

- **Final Round:** Completes the decryption by applying Inverse ShiftRows, Inverse SubBytes, and AddRoundKey, recovering the plaintext.
6. **NIST Compliance:** By adhering strictly to the NIST FIPS 197 guidelines, the implementation ensures it meets security requirements and is compatible with other standard AES implementations.

This project demonstrates a secure and efficient implementation of AES with a 128-bit key, showcasing the practical application of cryptographic principles in software development.

4 Code Structure and Modularity

The code for the AES implementation is organized with a focus on clarity, modularity, and adherence to cryptographic standards. The structure of the code is designed to make the implementation easy to understand, maintain, and extend.

1. **Header File (AES.h):** This file contains the declarations of the functions and constants used in the AES implementation. It serves as the interface for the AES algorithm, allowing other parts of the program to use the encryption and decryption functions without needing to understand the underlying details. The header file includes definitions for the S-Box, the inverse S-Box, and other constants necessary for the AES transformations.
2. **Source File (cipher.c):** This file contains the core implementation of the AES algorithm, including both the encryption and decryption processes. The code is divided into several well-defined functions, each responsible for a specific part of the AES algorithm:
 - **Key Expansion:** This function generates the round keys from the initial 128-bit cipher key. It includes the necessary rotations, substitutions, and XOR operations to produce the 11 round keys used throughout the encryption and decryption processes.
 - **AddRoundKey:** This function performs the XOR operation between the state matrix and the round key. It is used in both encryption and decryption to integrate the key into the data processing.

- **SubBytes and Inverse SubBytes:** These functions perform the byte substitution using the S-Box and inverse S-Box, respectively. They are essential for both encryption and decryption, ensuring the non-linearity of the transformation.
 - **ShiftRows and Inverse ShiftRows:** These functions handle the cyclic shifting of the rows in the state matrix. ShiftRows is used during encryption, while Inverse ShiftRows is used during decryption.
 - **MixColumns and Inverse MixColumns:** These functions mix the columns of the state matrix during encryption and decryption. MixColumns is applied during encryption, while Inverse MixColumns is used during decryption to reverse this step.
 - **Cipher and InvCipher:** The Cipher function orchestrates the entire encryption process, applying the series of AES transformations across 10 rounds. The InvCipher function does the same for the decryption process, applying the inverse transformations to recover the plaintext from the ciphertext.
3. **Modularity:** The modular nature of the code ensures that each function is focused on a single task, making the code more readable and easier to debug. This modularity also allows for easy modifications and enhancements. For example, if an alternative key expansion method or different S-Box were needed, these could be swapped out without altering the rest of the code.
 4. **Reusability:** The functions defined in the code are reusable across different parts of the program or in other projects. By separating the encryption and decryption logic into distinct, reusable functions, the code facilitates the integration of AES encryption into various applications that require secure data handling.

This structured and modular approach to implementing AES ensures that the code is both efficient and maintainable, allowing for easy updates and extensions while adhering to cryptographic best practices.

5 Testing and Validation

To ensure that the implementation is correct, it has been rigorously tested against known test vectors from the NIST standard. These test vectors provide expected outputs for given inputs and keys, allowing for validation of the implementation. By comparing the output of

the Cipher function with the expected ciphertext, we can confirm that the implementation adheres to the AES standard.

6 Security Considerations

The AES implementation in this project prioritizes security by strictly adhering to the NIST FIPS 197 standard. This adherence ensures robustness against cryptographic attacks, including:

1. **Strong Key Expansion:** The implementation generates 11 unique round keys from the 128-bit cipher key, making it difficult for attackers to reverse-engineer the encryption or deduce the original key.
2. **S-Box and Inverse S-Box:** The S-Box introduces non-linearity, essential for resisting linear and differential cryptanalysis. The corresponding Inverse S-Box ensures secure and accurate decryption.
3. **MixColumns Transformation:** This operation enhances diffusion by ensuring that each input byte influences the entire state matrix, making ciphertext changes unpredictable even with minor plaintext alterations.
4. **NIST Compliance:** Following the NIST guidelines ensures the implementation meets rigorous security standards, protecting against various attack vectors.

7 Optimization Techniques

To improve performance, the AES implementation incorporates several optimization techniques:

1. **Loop Unrolling:** Reducing loop iterations minimizes control overhead, speeding up critical encryption and decryption paths.
2. **Precomputed Tables:** Using precomputed S-Box and Inverse S-Box tables accelerates byte substitution operations, reducing computational complexity.
3. **Efficient Key Expansion:** Optimizations in the key expansion process reduce redundant calculations and memory usage, making the process faster and more efficient.

4. **Memory Optimization:** The implementation minimizes memory usage by optimizing data structures and reducing temporary variables, making it suitable for memory-constrained environments.
5. **Reduced Branching:** Minimizing conditional branches enhances execution speed by reducing pipeline stalls, particularly on modern CPU architectures.

These optimizations ensure the AES implementation is both secure and efficient, balancing the need for robust encryption with the demands of real-time performance. The code is optimized for speed and resource efficiency, making it suitable for a wide range of applications, including those with stringent performance requirements.

8 Conclusion

This project successfully demonstrates the implementation of the Advanced Encryption Standard (AES) using a 128-bit key, guided by the rigorous specifications of the NIST FIPS 197 standard. The implementation effectively combines security with performance through careful adherence to cryptographic standards and the application of various optimization techniques. By ensuring a strong key expansion process, utilizing non-linear transformations like the S-Box, and optimizing the code for efficiency, the project delivers a robust and secure encryption solution. This implementation not only meets the necessary security requirements but also proves to be efficient, making it suitable for real-time applications and environments with limited resources. The project highlights the practical application of cryptographic theory in real-world programming, emphasizing the importance of standardization and optimization in secure software development.